ECE449 Final Demo implementation & rubrics

Lehung Nguyen

Insights

- A fundamental constraint of 16-bit CPUs: maximum signed 16-bit integer is $32{,}767$ (or $65{,}535$ unsigned). Factorials grow exponentially; $8! = 40{,}320$ (breaks the signed limit) and $9!$ (breaks the 16-bit threshold)
- *testFact.asm:* ideal conditions. *testFactOverflow.asm:* catastrophic mathematical failures
- Hardware/Software Co-Design: for *testFactOverflow.asm* software cannot catch the error without the ALU being physically modified to generate the flag, and ISA needs to be updated so the software can read the flag (via the new *BRR.overflow* instruction)

How to handle BRR.Overflow

- Implement BRR.Overflow requires HW/SW co-design

1. Modify ALU:
   - By default, ALU multiplies two 16-bit numbers and immediately truncates the result to 16 bits, ignoring any bits that spill over. Need modify ALU.vhd to detect this spill.
   - Inside your ALU process, perform the multiplication using a 32-bit variable
   - Check if that 32-bit result falls outside the safe 16-bit signed range (> 32,767 or < -32,768)
2. Route the flag through the pipeline:
   - Similar to *flag_z* and *flag_n*, *flag_v* should be routed correctly.
   - If CPU evaluates branches in the Execute stage, flag can be read directly from ALU.
   - If CPU evaluates branches in a dedicated status register or a later stage, should add *flag_v* to ID/EX and EX/MEM pipeline registers so the flag travels alongside the instruction.

How to handle BRR.Overflow

3. Modify the Controller

- *Controller.vhd* needs to understand BRR.Overflow instruction (a variation of Format B branching instructions)
- Add a branch evaluation logic block: When the Controller sees the *BRR.overflow* opcode, it must check the *flag_v* wire

4. Calculate the Branch Target

- If the Controller sees the BRR.overflow instruction AND the *flag_v* is currently '1', it triggers a branch.
- Instruction should contain a relative displacement value (e.g., jump forward 5 lines)
- Hardware should take the current Program Counter (PC) address, add the displacement value, and route this new address to the PC's *next_pc* input, jumping over the standard loop code.
- If *flag_v* is '0', the instruction is ignored, and the PC simply increments to the next line as usual.

How to handle BRR.Overflow

3. Write the Assembly Code (Software Handling)

- Once the hardware supports it, you can utilize it in *TestFactOverflow.asm* script to safely catch the error.
- Place the *BRR.overflow* instruction immediately after the multiplication. If the hardware flag trips, the CPU will jump to your dedicated error-handling block.

rgence.m | 2-bootloader.asm | testFactOverflow.asm | testFactOverflow_BRROverflow.asm

```
LOOP:
    MUL r1, r1, r0              ; Perform the multiplication
    BRR.overflow OVERFLOW       ; HARDWARE CHECK: Did it exceed 16 bits?
                                ; If YES, jump to OVERFLOW label. If NO, continue

    SUB r0, r0, r5              ; Normal loop continues...
    SUB r4, r0, r6
    TEST r4
    BRR.N PRINT
    BRR LOOP

OVERFLOW:
    LOADIMM.upper 0x00          ; Error handler: Load 0 into a register
    LOADIMM.lower 0x00
    MOV r1, r7
    OUT r1                      ; Output 0 to indicate a mathematical failure
    BRR START                   ; Reset the program and wait for a new input
```

1. Basic factorial & loop (testFact.asm) (40P)

- Verify the CPU's ability to execute a mathematical loop, manage register states, and handle I/O ports.

Criteria	Point	Verified in the Simulation / Waveform
1.1 Register Initialization	10	*Verify:* LOADIMM and MOV correctly initialize r5 = 1 and r1 = 1. The instruction SHL r6, 1 correctly shifts r6 (which was 1) to become 2.
1.2 Pipeline Arithmetic (MUL, SUB)	15	*Verify:* Inside the loop, MUL r1, r1, r0 correctly computes the accumulating product, and SUB r0, r0, r5 correctly decrements the input counter by 1 each cycle. *Note:* Tests the ALU's arithmetic accuracy and ensures there are no Read-After-Write (RAW) data hazards when feeding the result of a subtraction directly into the next calculation.
1.3 Conditional Branching (TEST, BRR.N)	15	*Verify:* SUB r4, r0, r6 followed by TEST r4 correctly evaluates if the counter has dropped below 2. If r4 is negative, BRR.N PRINT must successfully jump out of the loop. *Note:* Validates the generation of the Negative (N) status flag in the ALU and the Control Unit's ability to calculate relative jump addresses based on that flag.

2. HW Overflow Implem. (testFactOverflow.asm) (45P)

- Verify hardware is modified to detect multiplication overflow and updated the assembly code to utilize this new feature

Criteria	Point	Verified in the Simulation / Waveform
2.1 Hardware Overflow Detection (ALU)	15	*Verify:* When the factorial exceeds 16 bits (e.g., computing 9! = 362,880), the ALU must assert a custom overflow flag. *Note:* The ALU multiplier must be physically wired to detect when bits carry over past the 16th bit.
2.2 Custom Branch Instruction (BRR.overflow)	15	*Verify:* Need to uncomment and implement the BRR.overflow OVERFLOW instruction (or equivalent). The Control Unit must correctly decode this new opcode/condition and vector the PC if the overflow flag is high. *Note:* Extend the Instruction Set Architecture (ISA) and modify the Controller's Finite State Machine to recognize new hardware flags.
2.3 Overflow Error Handling	15	*Verify:* If an overflow occurs, the program must branch to a designated overflow handling section and push the value 0 to the OUT port. *Note:* Validates that the software correctly catches the hardware exception and executes the requested fail-safe behavior instead of outputting garbage data.

3. Input/Output and System Stability (15P)

- Verify the CPU can successfully interact with the external environment continuously.

Criteria	Point	Verified in the Simulation / Waveform
3.1 I/O Port Functionality	10	*Verify:* IN r0 successfully loads a value from the external input port. OUT r1 successfully pushes the final factorial result to the external output port. *Note:* Confirms the memory-mapped I/O routing is properly decoding addresses to interface with external hardware (like switches or LEDs).
3.2 Continuous Execution (BRR START)	5	*Verify:* After printing the result, the unconditional branch BRR START successfully loops the entire program back to the beginning to wait for a new input. *Note:* Ensures the Program Counter can jump backwards large distances without faulting, allowing the CPU to run continuously.